DevSecOps

Building Security into the Way You Already Build Software

AGENDA

What We'll Cover Today

Why DevSecOps

Shifting security left in the SDLC

Secure Coding Basics

OWASP Top 10 and common mistakes

Pipeline Security Tools

SAST, SCA, DAST, container & IaC scanning

Live Pipeline Demo

Watching a scanner catch a real issue

Cloud & Container Security

IAM, least privilege, safe defaults

Logging & Response

Detecting and reacting to incidents

Security Used to Be a Final Gate

The Old Way

- ✗ Security team reviews code right before release
- ✗ Vulnerabilities found weeks or months after they were written
- ✗ Fixes are expensive and delay the launch
- ✗ Developers see security as "someone else's job"
- ✗ Slow, manual, and easy to skip under deadline pressure

The DevSecOps Way

- ✓ Security checks run automatically, every commit
- ✓ Issues are caught in minutes, while the code is fresh
- ✓ Fixing early is cheap; fixing late is costly
- ✓ Every developer owns security in their own code
- ✓ Fast, automated, and built into the pipeline

WHY DEVSECOPS

What Is DevSecOps?

DevSecOps integrates security practices into every stage of the DevOps lifecycle — instead of treating it as a separate, final step.

SHIFT LEFT

Plan → **Code** → **Build** → **Test** → **Release** → **Deploy** → **Operate** → **Monitor**

Security activities move earlier — into planning and coding — instead of waiting until release.

People

Every developer shares ownership of security — not just a security team.

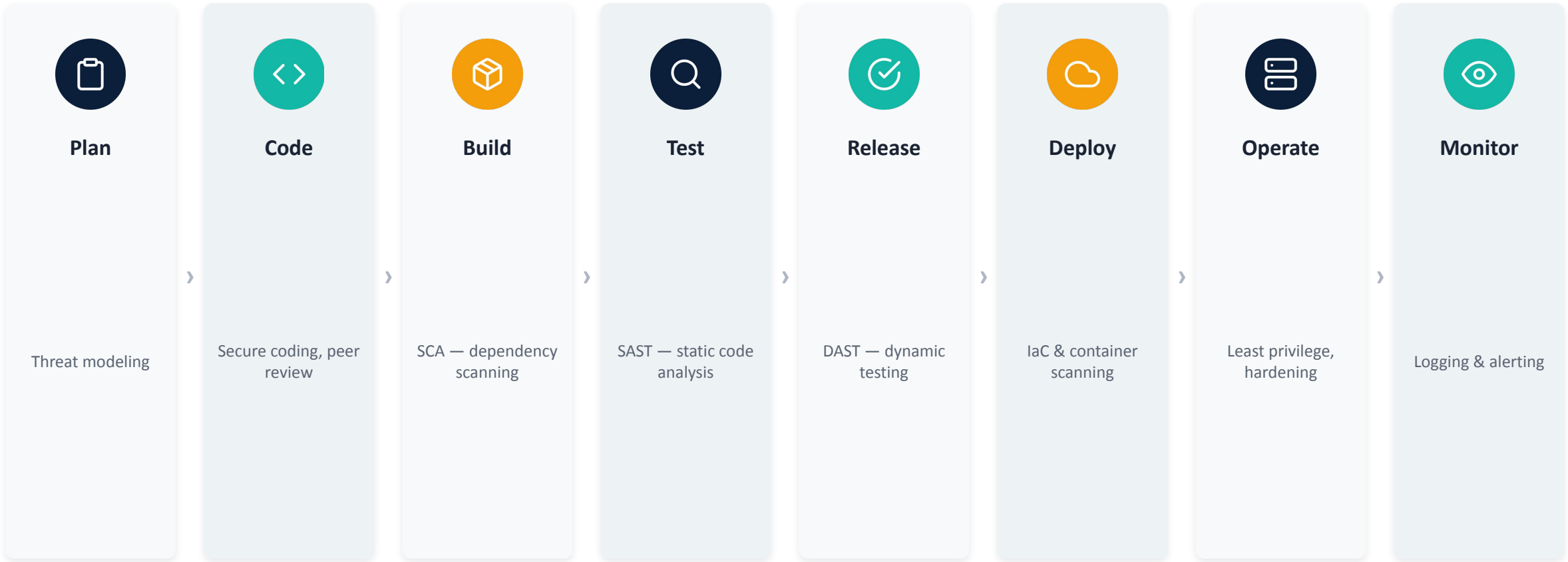
Process

Security checks are continuous and repeatable, not a one-time review.

Automation

Tools run in the pipeline itself, giving instant feedback on every change.

Security Activities Across the Pipeline



Security isn't one step. It's a set of automated checks spread across the whole lifecycle.

SECURE CODING

The OWASP Top 10 (2025)

The industry-standard list of the most critical web application security risks

- **A01** Broken Access Control
- **A02** Security Misconfiguration
- **A03** Software Supply Chain Failures
- **A04** Cryptographic Failures
- **A05** Injection
- **A06** Insecure Design
- **A07** Authentication Failures
- **A08** Software or Data Integrity Failures
- **A09** Security Logging & Alerting Failures
- **A10** Mishandling of Exceptional Conditions

Two Vulnerabilities Every Developer Should Recognize

A01 · Broken Access Control

A user can change a URL like /profile/677 to /profile/678 and view someone else's data because the server never checked ownership.

```
// Always verify on the server

if (record.ownerId !== currentUser.id) {

  return res.status(403).end();

}
```

Fix: enforce access checks server-side, deny by default, never trust client-side logic.

A05 · Injection

Untrusted input is concatenated straight into a query or command, letting an attacker change what actually gets executed.

```
// Risky: string concatenation

"SELECT * FROM users WHERE id=" + id

// Safer: parameterized query

db.query("...WHERE id = ?", [id])
```

Fix: use parameterized queries or ORMs, and validate/escape all input.

SECURE CODING

The #1 Rookie Mistake: Hardcoded Secrets

API keys, passwords, and tokens committed to source control are found by attackers within minutes — and by scanners even faster.

Don't

```
const apiKey =

  "sk_live_51H8x...";
```

Hardcoded directly in the file, committed to Git, visible to anyone with repo access forever, even after deletion.

Do

```
const apiKey =

  process.env.STRIPE_KEY;
```

Loaded from environment variables or a secrets manager (e.g. AWS Secrets Manager, HashiCorp Vault) and scanned for with a tool like gitleaks or truffleHog.

PIPELINE TOOLING

Automated Scanners in the CI/CD Pipeline

Category	What It Checks	Example Tools
SAST	Scans source code for insecure patterns	*SonarQube, Semgrep*
SCA	Finds known vulnerabilities in dependencies	*Snyk, Dependabot, OWASP Dependency-Check*
DAST	Tests the running application from the outside	*OWASP ZAP*
Container	Scans container images for vulnerabilities	*Trivy, Grype*
IaC	Checks infrastructure config before it's deployed	*Checkov, tfsec*
Secrets	Catches committed passwords and API keys	*gitleaks, truffleHog*

Each scanner plugs into a pipeline stage and runs automatically on every commit or pull request.

A Few Habits That Prevent Most Cloud Incidents

Least Privilege (IAM)

Grant only the permissions a service or user actually needs — never use admin/root as a default.

Don't Run Containers as Root

Set a non-root user in your Dockerfile so a compromised container can't take over the host.

Use Minimal, Verified Base Images

Prefer official or distroless images; scan them before they ship (e.g. with Trivy).

Scan Infrastructure as Code

Catch open storage buckets, exposed ports, and missing encryption before you apply the change.

MONITORING & RESPONSE

You Can't Respond to What You Can't See

Logging & Monitoring

- Log security-relevant events: logins, permission changes, failed access attempts
- Centralize logs so they can't be deleted by an attacker covering their tracks
- Alert on anomalies — don't just collect logs nobody reads

If You Suspect an Incident

- Don't panic, and don't delete or "clean up" anything — you may destroy evidence
- Report immediately to your security/incident response contact
- Write down what you saw and when — timelines matter during investigation

WRAP-UP

Start Your Week Securely: 5 Things to Remember

- **Security is everyone's job** — not a separate team's problem to solve after you're done.
- **Automate the checks** — SAST, SCA, DAST, container & IaC scans belong in your pipeline, not in someone's memory.
- **Never commit secrets** — use environment variables or a secrets manager, and scan for leaks.
- **Enforce access control server-side** — never trust a check that only happens in the browser.
- **Log it so you can see it** — you can't investigate or respond to what was never recorded.

Thank You